Unit 1

Introduction to OOPs and Basics of C++
1.1 Basic concepts of OOPs
1.2 Benefits of OOPs
1.3 C++ Tokens, Variables, Constants and data types
1.4 Basic Input / Output Statements
1.5 Structure of a C ++ program
1.6 Scope Resolution Operator
1.7 Control Structure : Conditional Statements, LoopingStatements, Jumping Statements
1.8 Arrays

Benefits of OOPs

1. **Modularity**: OOP divides complex systems into smaller components, making the codebase easier to comprehend, create, and maintain.
2. **Reusability**: Inheritance allows code reuse, improving code quality and saving time.
3. **Encapsulation**: Protects data integrity and privacy by restricting direct access and allowing controlled access through methods.
4. **Flexibility and Scalability**: OOP enables easy addition and modification of features without impacting the entire codebase.
5. **Code Organization**: OOP promotes a structured approach, enhancing collaboration and code readability.
6. **Code Maintenance**: Changes and bug fixes can be made to specific objects or classes without affecting other parts of the system, reducing errors and improving debugging.
7. **Code Reusability**: OOP encourages the development of reusable code elements, saving time and improving system reliability.
8. **Better Problem Solving**: OOP models real-world systems, allowing developers to create intuitive solutions that closely mimic real-world circumstances.
9. OOP also improves data security and lowers the chance of data corruption by enclosing data and behavior within objects.

Token :-

A C++ program is composed of tokens which are the smallest individual unit. Tokens can be one of several things, including keywords, identifiers, constants, operators, or punctuation marks.
There are 6 types of tokens in c++:

1. Keyword
2. Identifiers
3. Constants
4. Strings
5. Special symbols
6. Operators

Keywords :-

In C++, keywords are reserved words that have a specific meaning in the language and cannot be used as identifiers. Keywords are an essential part of the C++ language and are used to perform specific tasks or operations.
Example:- do,int,while,for etc.

Identifiers :-

In C++, an identifier is a name given to a variable, function, or another object in the code. Identifiers are used to refer to these entities in the program, and they can consist of letters, digits, and underscores. However, some rules must be followed when choosing an identifier:

- The first character must be a letter or an underscore.
- Identifiers cannot be the same as a keyword.
- Identifiers cannot contain any spaces or special characters except for the underscore.
- Identifiers are case-sensitive, meaning that a variable named "Abc" is different from a variable named "abc".

  Example :- my_variable , student_name

Constants :-

A constant is a value that cannot be changed during the execution of the program.

Syntax :-

const data-type constant-name = value;

The value of a constant cannot be changed once it has been defined. Here is an example:

const double PI = 3.14159;

In this example, the constant PI is of type double and is initialized with the value of pi. You can then use the constant PI in your code wherever you need to use the value of pi.

String :-

A string is a sequence of characters that represents text. **Strings are commonly used in C++ programs to store and manipulate text data**.

To use strings in C++, you must include the <string> header file at the beginning of your program. This will provide access to the string class, which is used to create and manipulate strings in C++.

Example:
string message =”Hello all”;
Cout<<message;

Output :- Hello all

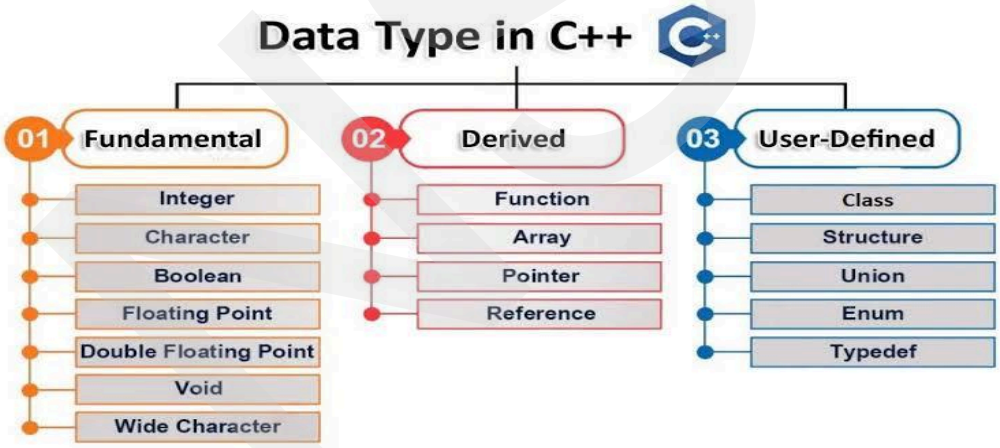
Special symbols :-

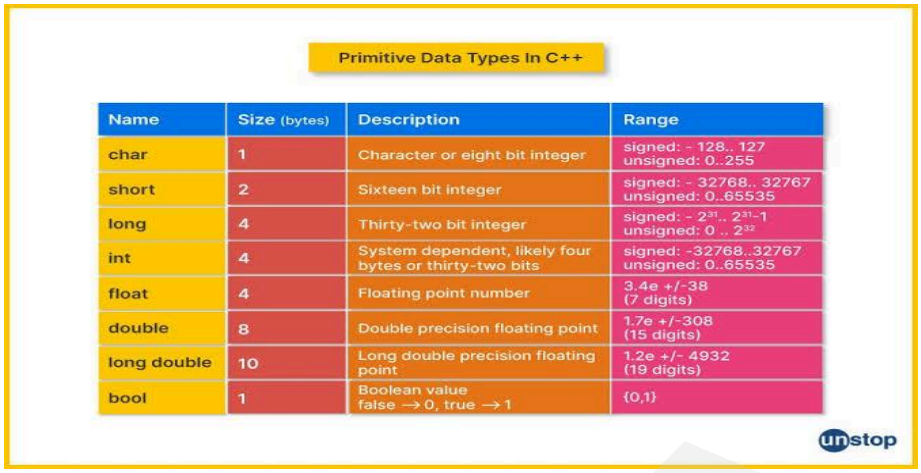
1. **Semicolon (;):** It is used to terminate the statement.
2. **Square brackets []:** They are used to store array elements.
3. **Curly Braces {}:** They are used to define blocks of code.
4. **Double-quote (“):** It is used to enclose string literals.
5. **Single-quote (‘):** It is used to enclose character literals.
6. **The ampersand (&)** : is used to represent the address of a variable.
7. **the tilde (~)** : is used to represent the bitwise NOT operator.
8. **asterisk (*)** : used as the multiplication and pointer operators.
9. **The pound sign (#)** : is used to represent the preprocessor directive, a special type of instruction processed by the preprocessor before the code is compiled.

C++ Data Types

Data types specify the type of data that a variable can store. Whenever a variable is defined in C++, the compiler allocates some memory for that variable based on the data type with which it is declared because every data type requires a different amount of memory.

C++ supports a wide variety of data types, and the programmer can select the data type according to the needs of the application.
In C++, data types are classified into the following types:





Fundamental Data Types

The data type specifies the size and type of information the variable will store:

1. Character Data Type (char)

- The **character data type** is used to store a single character.
- The keyword used to define a character is **char**.
- Its size is 1 byte and it stores characters enclosed in single quotes (‘ ‘).
- It can generally store upto 256 characters according to their ASCII codes.

Syntax:-

```
char variable-name;
```

Example:-

```
char test = 'h';
```

Or

```
#include <iostream>
using namespace std;
int main()
{
    // Character variable
      char c = 'A';
      cout << c;
      return 0;
}
```

Output :- A

2. Integer Data Type (int)

- The int keyword is used to indicate integers.
- Its size is usually 4 bytes. Meaning, it can store values from **-2147483648 to 2147483647**.

Syntax :- int variable-name;
Example :- int id = 5;

3. Boolean Data Type (bool)

- The **boolean data type** is used to store logical values: **true(1)** or **false(0)**.
- The keyword used to define a boolean variable is **bool**.
- Its size is 1 byte.

Syntax :- bool variable-name;
Example :- bool a = true;
Cout<<a;
Output:- 1

4. Floating Point Data Type (float)

- **Floating-point data type** is used to store numbers with decimal points.
- The keyword used to define floating-point numbers is **float**.
- Its size is 4 bytes (on 64-bit systems) and can store values in the range from **1.2E-38 to 3.4e+38.**

Syntax:- float variable-name;
Example:- float f = 36.5;

5. Double Data Type (double)

- The **double data type** is used to store decimal numbers with higher precision.
- The keyword used to define double-precision floating-point numbers is **double**.
- Its size is 8 bytes and can store the values in the range from **1.7e-308 to 1.7e+308**

Syntax :- double variable- name
Example:- double pi = 3.1415926535;

6. Void Data Type (void)

- The **void data type** represents the absence of value.
- We cannot create a variable of void type.
- It is used for pointers and functions that do not return any value using the keyword **void**.

Syntax:-
void function_Name ();

Example:-

```
#include <iostream >
using namespace std;
//Function with void return type
```

```
void hello()
{
        cout << "Hello, World!" << endl;
}

int main()
{
        hello();
        return 0;
}
```

Output: Hello

C++ Basic Input/Output :-

Header Files are those files that store predefined functions. It contains definitions of functions that you can include or import using a preprocessor directive(#). This preprocessor directive tells the compiler that the header file needs to be processed before the compilation.

C++ I/O occurs in streams, which are sequences of bytes.
If bytes flow from a device like a keyboard, a disk drive, or a network connection etc. to main memory, this is called **input operation** and if bytes flow from main memory to a device like a display screen, a printer, a disk drive, or a network connection, etc., this is called **output operation**.
I/O Library Header Files

There are following header files important to C++ programs −

Sr.No	Header File & Function and Description
1	**<iostream>** **It contains declarations of input and output operations using streams such as** This file defines the **cin, cout, etc.** objects, which correspond to the standard input stream, the standard output stream.
2	**<fstream>** This file declares services for user-controlled file processing. We will discuss it in detail in the File and Stream related chapter.

The Standard Output Stream (cout)

The predefined object **cout** is an instance of **ostream** class.
The cout object is said to be "connected to" the standard output device, which usually is the display screen.
The **cout** is used in conjunction with the **stream insertion operator**, which is written as << which are two less than signs as shown in the following example.
Example

```
#include <iostream>
 using namespace std;
 int main()
 {
         char str[] = "Hello C++";
        cout << "Value of str is : " << str << endl;
}
```

When the above code is compiled and executed, it produces the following result
Value of str is : Hello C++

The C++ compiler also determines the data type of variable to be output and selects the appropriate stream insertion operator to display the value. The << operator is overloaded to output data items of built-in types integer, float, double, strings and pointer values.
The insertion operator << may be used more than once in a single statement as shown above and **endl** is used to add a new-line at the end of the line.

The Standard Input Stream (cin)

The predefined object **cin** is an instance of **istream** class. The cin object is said to be attached to the standard input device, which usually is the keyboard.
The **cin** is used in conjunction with the **stream extraction operator,** which is written as >> which are two greater than signs as shown in the following example.
Example

```
#include <iostream>
 using namespace std;
 int main()
{
         char name[50];

         cout << "Please enter your name: ";
         cin >> name;
           cout << "Your name is: " << name << endl;

 }
```

When the above code is compiled and executed, it will prompt you to enter a name. You enter a value and then hit enter to see the following result −

Please enter your name: hello
Your name is: hello

The C++ compiler also determines the data type of the entered value and selects the appropriate stream extraction operator to extract the value and store it in the given variables.

Structure of a program :-

Program	Output
// my first program in C++ **#include <iostream>** **int main()** **{** ** std::cout << "Hello World!";** **}**	**Hello World!**

The left side shows the C++ code for this program. The right side shows the result when the program is executed by a computer.

Let's examine this program line by line:

Line 1: // my first program in C++

Two slash signs indicate that the rest of the line is a comment inserted by the programmer but which has no effect on the behavior of the program. Programmers use them to include short explanations or observations concerning the code or program.

Line 2: #include <iostream>

include in that # is a preprocessor which instruct Compiler to load the file which is in <> before starting the compile process because some functions used in the program are written in that file.

Line 3: A blank line.

Blank lines have no effect on a program. They simply improve readability of the code.

Line 4: int main ()

The function named **main** is a special function in all C++ programs; it is the function called when the program is run.

The execution of all C++ programs begins with the main function, regardless of where the function is actually located within the code.

Lines 5 and 7: { and }

The open brace ({) at line 5 indicates the beginning of **main's** function definition, and the closing brace (}) at line 7, indicates its end. Everything between these braces is the function's body that defines what happens when main is called. All functions use braces to indicate the beginning and end of their definitions.

Line 6: std::cout << "Hello World!";

This statement has three parts: First, std::cout, which identifies the **st**andar**d c**haracter **out**put device (usually, this is the computer screen). Second, the insertion operator (<<), which indicates that what follows is inserted into std::cout. Finally, a sentence within quotes ("Hello world!"), is the content inserted into the standard output.

Note :- The statement ends with a semicolon (;).
Space in output is given by cout<<" " or <<end l or <<"\n"

Conditional Statements in C++

Conditional statements execute blocks of code depending on the condition.Condition is written in if and else statements. If the condition is true then if block is executed otherwise else block is executed.

Types of Conditional Statements:

In C++, there are primarily three types of conditional statements:

1.If Statement:

The if statement is the most basic conditional statement. It allows you to execute a block of code only if a given condition is true.

Syntax :-

```
if (condition)
 {
// Code to be executed if the condition is true
}
```

Example : program to check a number is positive :-

```
#include <iostream>
using namespace std;
int main() {
int number;
cout << "Enter a number: ";
cin >> number;
```

```
if (number > 0) {
cout << "The number is positive." << endl;
}
return 0;
}
```

In this example, the code inside the if block will only be executed if the condition number > 0 is true.

2. if else statement:

The else statement is used with the if statement.

Here the condition of if block is checked and if that condition is false then the statement in else block is executed.

This example checks the number is positive or negative:
Here if the number entered by the user is number >0 then if block is executed and if number < 0 then else block is executed.

```
#include <iostream>
using namespace std;
int main() {
int number;
cout << "Enter a number: ";
cin >> number;
if (number > 0)
{
cout << "The number is positive." << endl;
}
 else
{
cout << "The number is non-positive (zero or negative)." << endl;
}
return 0;
}
```

3. if else if Statement:

When you have multiple conditions to check, you can use else if statements. They allow you to test different conditions one by one and execute the corresponding block of code for the first true condition found.

Here's an example which checks a number is positive, negative, or zero:

```
#include <iostream>
using namespace std;
int main()
{
```

```
    int number;
    cout << "Enter a number: ";
    cin >> number;

    if (number > 0)
    {
        cout << "The number is positive." << endl;
    }
    else if (number < 0)
    {
        cout << "The number is negative." << endl;
    }
    else
    {
        cout << "The number is zero." << endl;
    }

return 0;
}
```

Loop in C++

Loops allow us to execute a specific block of code repeatedly.

In programming, a loop is a control structure that repeatedly executes a block of code until a specified condition is true. It saves developers from writing the same code repeatedly , making their programs more efficient and manageable.

Types of Loop Statements:

C++ offers three primary types of loops:

1. **For Loop**
2. **While Loop**
3. **Do-While Loop**

1.For Loop:

The for loop is ideal when you know the exact number of iterations required.

It consists of three main parts:

Syntax:-

```
for (initialization; condition;increment/decrement)
{
// Code to be executed repeatedly or body of loop
}
```

Here
initialization - initializes variables and is executed only once

- condition - if true, the body of for loop is executed
  if false, the for loop is terminated
- update - updates the value of initialized variables and again checks the condition

Example :- Printing Numbers From 1 to 5.

```
#include <iostream>
using namespace std;
int main() {
      for (int i = 1; i <= 5; ++i) {
      cout << i << " ";
   }
   return 0;
}
```

Output :
1 2 3 4 5

2.While Loop:

A while loop is used when you need to execute a block of code as long as a certain condition remains true.

Here,

- A while loop evaluates the condition
- If the condition evaluates to true, the code inside the while loop is executed.
- The condition is evaluated again.
- This process continues until the condition is false.
- When the condition evaluates to false, the loop terminates.

Syntax :-

```
While (condition)
{
   // body of loop
}
```

Here's an example:

```
// C++ Program to print numbers from 1 to 5
#include <iostream>
using namespace std;
int main()
 {
          int i = 1;
          // while loop from 1 to 5
          while (i <= 5) {
          cout << i << " ";
           ++i;
```

```
        return 0;
}
```
Output:- 1 2 3 4 5

3. Do-While Loop:

A do-while loop is similar to a while loop, with one key difference: it always **executes the code block at least once, even if the condition is false.**

Syntax :-

```
do
{
        // Code to be executed repeatedly
} while (condition);
```

Here,

- The body of the loop is executed at first. Then the condition is evaluated.
- If the condition evaluates to true, the body of the loop inside the do statement is executed again.
- The condition is evaluated once again.
- If the condition evaluates to true, the body of the loop inside the do statement is executed again.
- This process continues until the condition evaluates to false. Then the loop stops.

```
// C++ Program to print numbers from 1 to 5
#include <iostream>
using namespace std;
int main()
{
  int i = 1;
   // do...while loop from 1 to 5
   do
{
      cout << i << " ";
      ++i;
 }while (i <= 5);
   return 0;
}
```
Output

1 2 3 4 5

Array:-

An array is a data structure that is used to store multiple values of similar data types in a contiguous memory location.

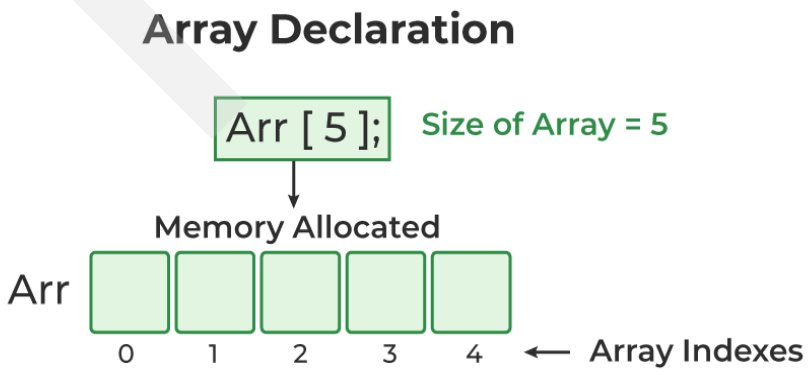
Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

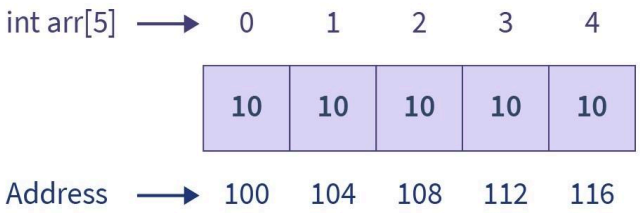
Properties of Array in C++

1. Arrays store data of the same data type in a contiguous memory location.
2. Indexing begins at 0, with the first element at the 0th index.
3. Elements are accessed using their indices.
4. Array size remains constant once declared.
5. Arrays can have multiple dimensions for complex data structures.
6. The sizeof operator determines the number of elements in an array.
7. Size of elements in an array

To declare an array, define the variable type, specify the name of the array followed by **square brackets** and specify the number of elements it should store:

Syntax:-
data_type array_name[Size_of_array];
Example
int arr[5];
Or
int arr[5] = {1, 2, 3, 4, 5};





SCALER Topics

```
//  C++ Program to Illustrate How to Access Array Elements

#include <iostream>
using namespace std;

int main()
{

    int arr[3];

    // Inserting elements in an array
    arr[0] = 10;
    arr[1] = 20;
    arr[2] = 30;

    // Accessing and printing elements of the array
    cout << "arr[0]: " << arr[0] << endl;
    cout << "arr[1]: " << arr[1] << endl;
    cout << "arr[2]: " << arr[2] << endl;

    return 0;
}
```

Output

```
arr[0]: 10
arr[1]: 20
arr[2]: 30
```